FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Gavryl Garlitos
Section:	An43
Professor:	Crisola G. Tan
Date:	October 13 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)
        self.path.append(self.position)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
```

```
    def setup(self):
        # 1. Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

          # 2. Assign random positions and initialize path history BEFORE
adding to grid
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))
            agent.path = [agent.position]

        # 3. Create grid and add agents
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation with Path Tracking -----------
fig, ax = plt.subplots()
ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True)
```

```
# Scatter plot for current positions and lines for paths
scat = ax.scatter([], [], s=100, color='red', zorder=3, label="Agents")
line_plots = [ax.plot([], [], lw=1.5, alpha=0.6)[0] for _ in
range(model.p.agents)]

def update(frame):
    model.step()
    # Update current positions
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))

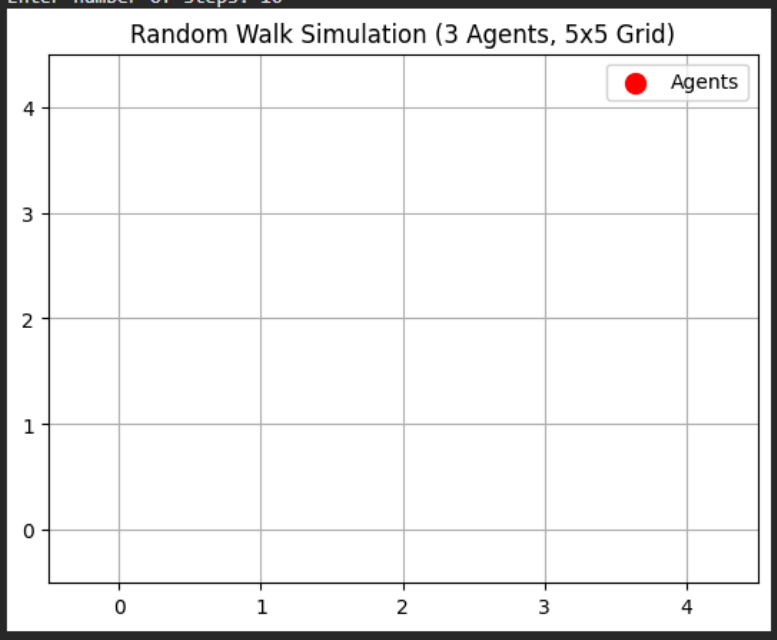
    # Update trail/paths for each agent
    for i, agent in enumerate(model.agents):
        px = [pos[0] for pos in agent.path]
        py = [pos[1] for pos in agent.path]
        line_plots[i].set_data(px, py)

    return [scat] + line_plots

ani = animation.FuncAnimation(fig, update, frames=model.p.steps,
interval=200, blit=False, repeat=False)
plt.title(f"Random Walk Simulation ({num_agents} Agents,
{grid_size}x{grid_size} Grid)")
plt.legend(loc='upper right')
plt.show()

# -------- Final Position Analysis Printout -----------
print("\n--- Final Positions of Agents ---")
for i, agent in enumerate(model.agents):
    print(f"Agent {i+1} Final Position: {agent.position}")
```

```
Enter number of agents: 3
Enter grid size (e.g., 10 for 10x10): 5
Enter number of steps: 10
```




```
--- Final Positions of Agents ---
Agent 1 Final Position: (0, 0)
Agent 2 Final Position: (3, 1)
Agent 3 Final Position: (0, 2)
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. When the agent density is extremely high relative to the available grid space, agents frequently crowd into the same coordinates or block each other's boundaries. In models with collision or avoidance rules, this leads to heavy congestion and restricted movement, whereas in simple overlapping random walks, multiple agents will stack on top of identical grid cells.

2. Increasing the number of steps allows agents more time to diffuse outward from their initial random starting points. As the simulation duration grows, the spatial distribution of the agents becomes wider, covering a larger portion of the grid due to the cumulative nature of random movement.
3. No, agents never move outside the grid. This behavior is intentionally prevented by boundary restriction logic in the step() function (x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))), which clamps the coordinates to stay within minimum and maximum grid boundaries.
4. Random walk models serve as the foundational building blocks for simulating various real-world systems:

- **Diffusion:** Models how gas molecules or liquids spread from areas of high concentration to low concentration.
- **Crowd Movement / Pedestrian Dynamics:** Simulates how people navigate spaces (like evacuating a building or walking through a terminal), especially when modified with directional preferences or obstacle avoidance.
- **Financial Markets:** Used in quantitative finance (such as the random walk hypothesis) to model the unpredictable fluctuations of stock prices over time.